

Problem Set 3

Tate Mason

May 10, 2026

1 Introduction

In problem set 3, I will be evaluating the expansion decisions in the context of Amazon's distribution network. The data given includes both observed data and perturbed data, where the perturbed data includes counterfactuals of entry decisions. I will be using this data familiarize myself with problems where partial identification is the aim, I will use the given moment inequalities to estimate the bounds on parameters γ and ω , where γ is the disutility of increased distance and ω is the parameter for population density.

2 Set Up and Accounting

For this problem, I had to create the expenditure functions e_{ijt} for each of the four modes of shopping: Amazon, taxed online, untaxed online, and in-store. The expenditures should sum to overall expenditure such that:

$$e = \sum_{j=0}^4 e_{ijt} \quad (1)$$

where e is the overall expenditure. I then had to calculate the mode specific expenditure, given by:

$$\begin{aligned} e_{i0t} &= \alpha_{i0t} \\ e_{i1t} &= \alpha_{i1t} + (1 - \sigma)(\log(1 + \tau_{i1t}) - \log(1 + \tau_{i0t})) \text{ where } \tau_{i1t} = \mathbf{1}\{FC_{it} = 1\} \cdot \tau_{i1t} \\ e_{i2t} &= \alpha_{i2t} + (1 - \sigma)(\log(1 + \tau_{i2t}) - \log(1 + \tau_{i0t})) \\ e_{i3t} &= \alpha_{i3t} + (1 - \sigma)(\log(1) - \log(1 + \tau_{i0t})) \end{aligned}$$

where α_{ijt} is the mode specific preference and τ_{ijt} is the tax rate for mode j in market i at time t . To recover the outside option's expenditure, I used the fact that the expenditures must sum to overall expenditure, so I calculated e_{i0t} as follows:

$$e_{i0t} = \frac{e}{1 + \sum_{j=1}^3 \exp(e_{ijt})} \quad (2)$$

with e corresponding to the data on average annual spending per county. From there, I was able to calculate the discounted sum of the following:

1. Gross Revenue:

$$R_t(a_t) = \sum_i M_{it} e_{i1t}(a_t)$$

Estimate: \$417,159,919,051

2. Labor Costs:

$$C_t^l(a_t) = \sum_i w_{it} \ell_{it}(a_t)$$

Estimate: \$7,046,187,457

3. Land Costs:

$$C_t^k(a_t) = \sum_i r_{it} k_{it}(a_t)$$

\$984,284,350

4. Shipping Distance: 15,790,629 county-miles

5. Effective Population Density: 180,286

6. Population Weighted Average Tax: 0.01

Now, when considering the perturbations, I calculated the mean change in the above variables as follows:

1. Change in Gross Revenue: $\approx$ -\$108,000,000
2. Change in Labor Costs: $\approx$ -\$42,000,000
3. Change in Land Costs: $\approx$ -\$5,000,000
4. Change in Shipping Distance: $\approx$ +2,000,000 county-miles
5. Change in Effective Population Density: $\approx$ -24,000
6. Change in Population Weighted Average Tax: $\approx$ +.0005

From these results, the perturbed location decisions decrease gross revenue, labor costs, and land costs, while increasing shipping distance, decreasing effective population density, and increasing the population weighted average tax (minorly). This leads to interesting tradeoffs, as the decrease in costs may be offset by the increase in shipping distance and tax rates alongside the already decreased revenue.

Next, I calculated the pairwise correlations between components of profit, given below:

	Revenue	Labor	Land	Shipping	Pop. Density	Tax Rate
Revenue	1.00	-0.25	-0.19	-0.11	-0.18	0.81
Labor	-0.25	1.00	0.93	0.11	0.19	-0.26
Land	-0.19	0.93	1.00	0.13	0.25	-0.20
Shipping	-0.11	0.11	0.13	1.00	0.13	-0.23
Pop. Density	-0.18	0.19	0.25	0.13	1.00	-0.07
Tax Rate	0.81	-0.26	-0.20	-0.23	-0.07	1.00

Table 1: Pairwise Correlations Between Components of Profit

Some interesting correlations to note are the strong positive correlation between revenue and tax rate, which may be due to the fact that higher tax rates are often found in more populous areas where revenue is higher. Further, the strong positive correlation between labor and land costs is expected, as these costs are often related to the size of the facility and the number of workers needed. The other correlations are mostly what I would expect, with negative correlations between revenue and costs, and positive correlations between costs and shipping distance/population density.

3 Estimating the Model

3.1 Isolating γ

To estimate γ , I used moment inequalities based on the following:

$$M_m(\omega, \gamma) = \frac{1}{S} \sum_s \bar{y}(a^*, a_s) - \gamma \frac{1}{S} \sum_s x^d(a^*, a_s) - \omega \frac{1}{S} \sum_s x^p(a^*, a_s) \geq 0 \quad (3)$$

where a^* is the observed entry decision and a_s are the perturbed entry decisions. I then calculated the bounds on γ by assuming $x^p(a^*, a_s) = 0$ and solving for γ such that the moment inequality holds. This gives us the following bounds on γ :

$$\gamma \in [-113.04, -57.59]$$

These bounds suggest that increased distance is actually a positive factor in the utility of the distribution centers, which is counterintuitive. This suggests an error in estimation, as we would definitely expect greater distance to result in greater costs, and thus less profit. To counter this, I then refine to groups of swaps that have large changes in distance but low tax changes. I did this because we are dealing with a distance parameter, but just conditioning on large distance changes would influence profit through other channels. So, conditioning on tax as well helps to control for

this. This gives us the following bounds on γ :

$$\gamma \in [-388.5, 171.05]$$

These estimates are mixed, as we see that the bounds on γ now include both negative and positive values, which is more in line with our expectations. However, the wide range of the bounds suggests that there is still a lot of uncertainty in the estimation of γ , which may be due to the fact that we are only using a subset of the data to estimate it.

Now, when we do not condition whatsoever, we get the following bounds on γ :

$$\gamma \in [-73.17, 26.28]$$

The bounds here are much tighter than my second set, and include both negative and positive values, which is more in line with our expectations. However, the bounds were tighter in the distance constrained only. This suggests that the conditioning on distance and tax may have been too restrictive, and that we may be better off using the full dataset to estimate γ . Here, the lower tail of the population density is definitely heavily influential in distance, as less populous areas are more likely to be farther away from distribution centers, so the distance parameter may be picking up some of the effects of population density, which is why the bounds are tighter when we do not condition on tax and distance.

3.2 Estimating γ and ω

```
#####
# PS3 Code      Econ 8220, Spring 2026 | Tate Mason
#####

import pandas as pd
import numpy as np

# =====
# Parameters
# =====
sigma    = 1.5      # demand elasticity (1 - sigma = -0.5)
beta     = 0.95     # discount factor
mu       = 0.15     # Amazon profit margin
base_yr  = 1999     # base year for discounting

# =====
# Step 1. Load and parse observed data
# =====
obs = pd.read_csv('../Data/data_observed.csv', sep='\t')

# Pivot so each county-year has one row with alpha for each mode (1, 2, 3)
# Long --> Wide: index = county-year, columns = mode, values = alpha
```

```

obs_piv = obs.pivot_table(
    index=['year', 'county'], columns='mode', values=['alpha'], aggfunc='first'
).reset_index()
obs_piv.columns = ['year', 'county', 'alpha1', 'alpha2', 'alpha3']

# Pull non-alpha columns from mode==1 rows (same across modes within county-year)
other = obs[obs['mode'] == 1][[
    'year', 'county', 'nb_hh', 'avg_spending_annual_county', 'tax_rate',
    'pop_density', 'rent', 'wage', 'min_dist0',
    'presence_state0', 'tot_empl0', 'tot_size0', 'presence_county0'
]].copy()

df = obs_piv.merge(other, on=['year', 'county'])

# =====
# Step 2. Compute spending by mode using the demand system
# =====
# Effective tax rates by mode:
# tau_0 = tax_rate (offline always taxed)
# tau_1 = tax_rate * presence_state0 (Amazon taxed only if FC in state)
# tau_2 = tax_rate (taxed online always)
# tau_3 = 0 (untaxed online never)
#
# Simplifications:
# j=2: tau_2 = tau_0 => log ratio = alpha_2 (tax terms cancel exactly)
# j=3: tau_3 = 0 => log ratio = alpha_3 + (1-sigma)*[log(1) - log(1+tau_0)]
# j=1: ratio depends on presence_state0

df['tau0'] = df['tax_rate']
df['tau1'] = df['tax_rate'] * df['presence_state0']

df['ratio1'] = np.exp(
    df['alpha1'] + (1 - sigma) * (np.log(1 + df['tau1']) - np.log(1 + df['tau0']))
)
df['ratio2'] = np.exp(df['alpha2']) # tau2 = tau0, difference is zero
df['ratio3'] = np.exp(
    df['alpha3'] + (1 - sigma) * (np.log(1 + 0) - np.log(1 + df['tau0']))
)

# Solve for levels:
# e_total = e_0 * (1 + r1 + r2 + r3) => e_0 = e_total / (1 + r1 + r2 + r3)
# e_j = r_j * e_0
df['e0'] = df['avg_spending_annual_county'] / (1 + df['ratio1'] + df['ratio2'] + df['ratio3'])
df['e1'] = df['ratio1'] * df['e0']

# Sanity check
assert np.allclose(

```

```

    df['e0'] + df['e1'] + df['ratio2'] * df['e0'] + df['ratio3'] * df['e0'],
    df['avg_spending_annual_county']
), "Spending_shares_don't_sum_to_total_ _check_demand_system."

# =====
# Step 3. Observed quantities for each county-year
# =====
df['rev_o'] = df['nb_hh'] * df['e1'] # Amazon gross revenue
df['lab_o'] = df['wage'] * df['tot_empl0'] # labor cost
df['land_o'] = df['rent'] * df['tot_size0'] # land cost
df['pop_o'] = df['pop_density'] * df['presence_county0'] # effective pop density

df['disc'] = beta ** (df['year'] - base_yr)

# Population-weighted Amazon tax rate components
df['tau1'] = df['tax_rate'] * df['presence_state0']
df['nb_tau1'] = df['nb_hh'] * df['tau1']

def disc_sum(df, col):
    """Aggregate_col_across_counties_within_each_year,_then_discount_and_sum_over_
    years."""
    annual = df.groupby('year')[col].sum()
    return (annual * (beta ** (annual.index - base_yr))).sum()

obs_R = disc_sum(df, 'rev_o')
obs_L = disc_sum(df, 'lab_o')
obs_K = disc_sum(df, 'land_o')
obs_D = disc_sum(df, 'min_dist0')
obs_P = disc_sum(df, 'pop_o')

obs_nbhh_disc = disc_sum(df, 'nb_hh')
obs_tax_sum = disc_sum(df, 'nb_tau1')
obs_tax_rate = obs_tax_sum / obs_nbhh_disc

print("===_Observed_Discounted_Sums_===")
print(f"_Gross_Revenue:_{obs_R:>20,.2f}")
print(f"_Labor_Cost:_{obs_L:>20,.2f}")
print(f"_Land_Cost:_{obs_K:>20,.2f}")
print(f"_Shipping_Distance:_{obs_D:>20,.2f}_county-miles")
print(f"_Effective_Pop._Density:_{obs_P:>20,.2f}")
print(f"_Pop.-wtd_avg_Amazon_tax:_{obs_tax_rate:.4f}")

# =====
# Step 4. Load perturbation files and compute swap-level deltas
# =====
p_all = pd.concat(
    [pd.read_csv(f'../Data/perturb_data{i}.csv', sep='\t') for i in range(1, 5)],
    ignore_index=True

```

```

)
print(f"\nLoaded_{len(p_all):,}_perturb_rows_across_{p_all['swapid'].nunique()}_swaps.
")

# Merge observed quantities onto perturb rows
obs_cols = ['year', 'county',
            'alpha1', 'alpha2', 'alpha3',
            'nb_hh', 'avg_spending_annual_county', 'tax_rate',
            'pop_density', 'rent', 'wage',
            'min_dist0', 'presence_state0', 'tot_empl0', 'tot_size0', '
            presence_county0',
            'rev_o', 'lab_o', 'land_o', 'pop_o', 'disc', 'tau1']
pm = p_all.merge(df[obs_cols], on=['year', 'county'], how='left')

# Perturbed spending (e1 under new network)
pm['tau1_p'] = pm['tax_rate'] * pm['presence_state1']
pm['ratio1_p'] = np.exp(
    pm['alpha1'] + (1 - sigma) * (np.log(1 + pm['tau1_p']) - np.log(1 + pm['tax_rate']
    )))
)
pm['ratio2_p'] = np.exp(pm['alpha2'])
pm['ratio3_p'] = np.exp(
    pm['alpha3'] + (1 - sigma) * (np.log(1 + 0) - np.log(1 + pm['tax_rate'])))
)
pm['e1_p'] = (
    pm['avg_spending_annual_county'] /
    (1 + pm['ratio1_p'] + pm['ratio2_p'] + pm['ratio3_p'])
) * pm['ratio1_p']

# Perturbed quantities
pm['rev_p'] = pm['nb_hh'] * pm['e1_p']
pm['lab_p'] = pm['wage'] * pm['tot_empl1']
pm['land_p'] = pm['rent'] * pm['tot_size1']
pm['pop_p'] = pm['pop_density'] * pm['presence_county1']

# Discounted deltas: (observed - perturbed) * discount factor
for col, obs_c, pert_c in [
    ('rev', 'rev_o', 'rev_p'),
    ('lab', 'lab_o', 'lab_p'),
    ('land', 'land_o', 'land_p'),
    ('dist', 'min_dist0', 'min_dist1'),
    ('pop', 'pop_o', 'pop_p'),
]:
    pm[f'ddelta_{col}'] = (pm[obs_c] - pm[pert_c]) * pm['disc']

# For population-weighted tax rate
pm['delta_wtax'] = (pm['tau1'] - pm['tau1_p']) * pm['nb_hh'] * pm['disc']

```

```

# Aggregate deltas to swap level
swap = pm.groupby('swapid')[[
    'ddelta_rev', 'ddelta_lab', 'ddelta_land',
    'ddelta_dist', 'ddelta_pop', 'delta_wtax'
]].sum()

perturb_q1 = pd.DataFrame({
    'revenue':      obs_R      - swap['ddelta_rev'],
    'labor':        obs_L      - swap['ddelta_lab'],
    'land':         obs_K      - swap['ddelta_land'],
    'distance':     obs_D      - swap['ddelta_dist'],
    'pop_density':  obs_P      - swap['ddelta_pop'],
    'tax_rate':     (obs_tax_sum - swap['delta_wtax']) / obs_nbhh_disc,
}, index=swap.index)

print("\nMeans:")
print(perturb_q1.mean())
# =====
# Step 5. Build 10-column panel and estimation variables
# =====
panel = pd.DataFrame(index=swap.index)

# Columns 1-5: observed (constant across swaps)
panel['obs_rev'] = obs_R
panel['obs_lab'] = obs_L
panel['obs_land'] = obs_K
panel['obs_dist'] = obs_D
panel['obs_pop'] = obs_P

# Columns 6-10: perturbed = observed + (perturbed - observed)
panel['pert_rev'] = obs_R - swap['ddelta_rev']
panel['pert_lab'] = obs_L - swap['ddelta_lab']
panel['pert_land'] = obs_K - swap['ddelta_land']
panel['pert_dist'] = obs_D - swap['ddelta_dist']
panel['pert_pop'] = obs_P - swap['ddelta_pop']

panel['delta_rev'] = panel['obs_rev'] - panel['pert_rev']
panel['delta_lab'] = panel['obs_lab'] - panel['pert_lab']
panel['delta_land'] = panel['obs_land'] - panel['pert_land']
panel['delta_dist'] = panel['obs_dist'] - panel['pert_dist']
panel['delta_pop'] = panel['obs_pop'] - panel['pert_pop']

# Estimation variables from the revealed-preference inequality:
panel['y_tilde'] = mu * panel['delta_rev'] - panel['delta_lab'] - panel['delta_land']
panel['x_d'] = panel['delta_dist']
panel['x_p'] = panel['delta_pop']

# Perturbed population-weighted tax rate

```

```

panel['tax_rate_pert'] = (obs_tax_sum + swap['delta_wtax']) / obs_nbhh_disc

panel.to_csv('../Data/estimation_panel.csv')
print("\n10-column_estimation_panel_saved_to_estimation_panel.csv")
print(f"Number_of_swaps:{len(panel)}")

# =====
# Question 2. Pairwise Correlations
# =====
print("\n==_Question_2:_Pairwise_Correlations_==")
corr_df = panel[['pert_rev', 'pert_lab', 'pert_land', 'pert_dist', 'pert_pop', '
    tax_rate_pert']].rename(
    columns={
        'pert_rev':      'Revenue',
        'pert_lab':      'Labor',
        'pert_land':     'Land',
        'pert_dist':     'Distance',
        'pert_pop':      'Pop._Density',
        'tax_rate_pert': 'Tax_Rate'
    }
)
print(corr_df.corr().round(3).to_string())

# =====
# Exercise 3.1(a). Isolate gamma      conditioning on  $|x_p| < 25\text{th percentile}$ 
# =====
print("\n==_Exercise_3.1(a):_gamma_bounds,_conditioning_on_|x_p|<p25_==")

abs_xp = panel['x_p'].abs()
p25_xp = abs_xp.quantile(0.25)
sub_a = panel[abs_xp <= p25_xp]
pos_a = sub_a[sub_a['x_d'] > 0]    # observed has longer distance      UB on gamma
neg_a = sub_a[sub_a['x_d'] < 0]    # observed has shorter distance     LB on gamma

# Rearranging by sign of sum(x_d):
ub_a = pos_a['y_tilde'].sum() / pos_a['x_d'].sum()
lb_a = neg_a['y_tilde'].sum() / neg_a['x_d'].sum()

print(f"25th_percentile_of_|x_p|:{p25_xp:.2f}")
print(f"Swaps_used:{len(sub_a)}_total_({len(pos_a)}_with_x_d>0,{len(neg_a)}_with_
    x_d<0)")
print(f"Upper_bound_on_gamma_(x_d>0_group):{ub_a:.4f}")
print(f"Lower_bound_on_gamma_(x_d<0_group):{lb_a:.4f}")
print(f"Identified_interval:{min(lb_a,ub_a):.4f},{max(lb_a,ub_a):.4f}")

panel['delta_tax'] = panel['tax_rate_pert'] - obs_tax_rate # or however you stored it

```

```

abs_xd      = panel['x_d'].abs()
abs_dtax    = panel['delta_tax'].abs()

# Thresholds      tune these
xd_thresh   = abs_xd.quantile(0.50)
tax_thresh  = abs_dtax.quantile(0.25)

# Refined groups
pos_refined = panel[(panel['x_d'] > xd_thresh) & (abs_dtax <= tax_thresh)]
neg_refined = panel[(panel['x_d'] < -xd_thresh) & (abs_dtax <= tax_thresh)]

ub_refined = pos_refined['y_tilde'].sum() / pos_refined['x_d'].sum()
lb_refined = neg_refined['y_tilde'].sum() / neg_refined['x_d'].sum()

print("\n==_Refined_bounds_with_additional_tax_change_conditioning_==")
print(f"__Thresholds: __|x_d|_>_{xd_thresh:.2f}, __|delta_tax|_<=_{tax_thresh:.4f}")
print(f"__Swaps_used: __{len(pos_refined)}_with_x_d>_{xd_thresh:.2f}, __{len(neg_refined)}
    __with_x_d<_{-xd_thresh:.2f}")
print(f"__Upper_bound_on_gamma: __{ub_refined:.4f}")
print(f"__Lower_bound_on_gamma: __{lb_refined:.4f}")
print(f"__Identified_interval: __[{min(lb_refined, ub_refined):.4f}, __{max(lb_refined,
    ub_refined):.4f}]]")
# =====
# Exercise 3.1(b). Gamma bounds      all swaps, no conditioning
# =====
print("\n==_Exercise_3.1(b):_gamma_bounds,_all_swaps_==")

pos_b = panel[panel['x_d'] > 0]
neg_b = panel[panel['x_d'] < 0]
ub_b  = pos_b['y_tilde'].sum() / pos_b['x_d'].sum()
lb_b  = neg_b['y_tilde'].sum() / neg_b['x_d'].sum()

print(f"__Swaps: __{len(pos_b)}_with_x_d>0, __{len(neg_b)}_with_x_d<0")
print(f"__Upper_bound_on_gamma: __{ub_b:.4f}")
print(f"__Lower_bound_on_gamma: __{lb_b:.4f}")
print(f"__Identified_interval: __[{min(lb_b, ub_b):.4f}, __{max(lb_b, ub_b):.4f}]]")

```